

Fájlkezelés Pythonban: olvasás és írás

Jegyzet és feladatlap

Hogyan dolgozz ezzel a jegyzettel?

A jegyzet két részből áll. Az 1–4. fejezetben megtanulod, hogyan olvas a programod adatokat fájlból, és hogyan ír eredményt fájlba. Az 5–8. fejezet összetett feladatokat tartalmaz, amelyekben az adatok már fájlból érkeznek.

- **Ellenőrzés.** Minden feladatnál ott van, mit kell kapnod. Csak akkor lépj tovább, ha egyezik.
- **Hibakereső.** Ha valami nem működik, a jegyzet végén lévő táblázat segít.
- **Segítség.** Az 5–8. fejezet feladataihoz a jegyzet legvégén találsz útbaigazítást. Csak akkor lapozz oda, ha legalább öt percig próbálkoztál.

A C++-os társaid ugyanezeket a fájlokat és feladatokat kapták, ugyanezzel a számozással. A feladatokhoz kilenc adatfájl tartozik:

Fájl	Mi van benne?	Fejezet
szamok.txt	néhány egész szám	1. és 4.
nevsor.txt	nevek, soronként egy	2.
diakok.txt	diákok adatai: <code>nev;kor;atlag</code>	2. és 4.
termek.txt	termékek, az első sorban a darabszám	3.
meresek.txt	mérések, az első sorban fejléc	3.
lepesek.txt	egy hónap napi lépésszámai	5.
hivasok.txt	egy ügyfélszolgálat hívásai	6.
verseny.txt	egy futóverseny eredményei	7.
vasarlasok.txt	egy büfé heti forgalma	8.

1. Olvasás fájlból

Egy fájl tartalmát az `open` függvénnyel éred el. A beolvasott tartalom mindig szöveg, ugyanúgy, ahogy az `input()` eredménye.

Kísérlet – hol fut a programom?

A program csak akkor találja meg az adatfájlt, ha az a megfelelő mappában van. Hogy melyik az, azt ezzel a programmal derítheted ki. (Hogy pontosan hogyan működik, az a 4. fejezetben derül ki.)

```
with open("itt_vagyok.txt", "w") as ki:
    print("Ebben a mappában fut a programod.", file=ki)
```

Futtasd, aztán keresd meg a gépeden az `itt_vagyok.txt` fájlt (általában a programod mellett lesz). Ahol megtaláltad, **oda másold be mind a kilenc adatfájlt.**

Próbáld ki - az első beolvasás

A `szamok.txt` tartalma (nyisd meg, és nézd meg):

```
12 7 25 4
18 31
9 16 22 3 40 5
```

```
with open("szamok.txt") as be:
    tartalom = be.read()

print(tartalom)
```

Nézzük meg az új részeket:

- `open("szamok.txt")` - megnyitja a fájlt **olvasásra**.
- `as be` - a megnyitott fájl ezen a néven érhető el. A nevet te választod.
- `with ... :` - a behúzott rész alatt a fájl nyitva van. A blokk végén a Python magától bezárja.
- `be.read()` - a fájl teljes tartalmát egyetlen sztringként adja vissza.

Kísérlet - ha a program nem találja a fájlt

Írd el a fájl nevét a programban: `"szamoc.txt"`. Futtasd, és olvasd el a hibaüzenet utolsó sorát. Utána javítsd vissza.

Figyelem! Ha `FileNotFoundError` hibát kapsz, két dolgot nézz meg: jól írtad-e a fájl nevét, és jó mappában van-e a fájl.

Tippelj - szöveg vagy szám?

A `split()` a szóközőknél és a sortöréseknél is darabol. Mit ír ki ez a részlet?

```
darabok = tartalom.split()

print(darabok)
print(len(darabok))
print(darabok[0] + darabok[1])
```

Az utolsó sor meglepő lehet. A fájlból minden szöveggként érkezik: ha számolni akarsz vele, az `int()` függvénnyel át kell alakítanod.

Próbáld ki - az összes szám beolvasása

```
with open("szamok.txt") as be:
    tartalom = be.read()

szamok = []
for resz in tartalom.split():
    szamok.append(int(resz))

print(len(szamok), "szamot olvastam be.")
```

Figyeld meg a fájl tartalmát: a számok három sorban, szabálytalanul állnak. Zavarta ez a `split()` metódust?

Mostantól az adatok a `szamok` listában vannak, és mindent megtehetsz velük, amit a listás jegyzetben tanultál.

Feladatok

1.1. feladat ★☆☆ - Az első három

Írd ki a `samok.txt` első három számát egy sorba.

Ellenőrzés: 12 7 25

1.2. feladat ★☆☆ - Darab és összeg

Olvasd be az összes számot egy listába. Írd ki, hány szám van, és mennyi az összegük.

Ellenőrzés: 12 szám, az összegük 192.

1.3. feladat ★☆☆ - Legnagyobb

Írd ki a fájl legnagyobb számát.

Ellenőrzés: 40.

1.4. feladat ★☆☆ - Párosak

Hány páros szám van a fájlban?

Ellenőrzés: 6.

2. Sorok olvasása

Ha egy sor összetartozó adatokat tartalmaz, soronként kell olvasnod. Pythonban a megnyitott fájlra `for` ciklussal lehet soronként végigmenni.

Tippelj - szavak vagy sorok?

A `nevsor.txt` tartalma:

```
Kiss Anna  
Nagy Bence  
Toth Csilla Reka  
Szabo Dani  
Varga Emma  
Kovacs Ferenc Pal
```

Hányat ír ki ez a program?

```
with open("nevsor.txt") as be:  
    tartalom = be.read()  
  
print(len(tartalom.split()))
```

Próbáld ki - soronként

```
with open("nevsor.txt") as be:  
    for sor in be:  
        print(sor)
```

A ciklusban a `sor` változó sorban a fájl egy-egy sorát kapja meg. Nézd meg jól a kimenetet: mi a furcsa benne?

Kísérlet - mi van a sor végén?

```
with open("nevsor.txt") as be:  
    for sor in be:  
        print(len(sor), sor)
```

Számold meg: a `Kiss Anna` hány karakter? Mit ír ki a program?

A fájlból beolvasott sor végén ott van a sortörés is, láthatatlan karakterként. Ezt a `strip()` vágja le:

```
sorok = []
with open("nevsor.txt") as be:
    for sor in be:
        sorok.append(sor.strip())

print(len(sorok), "sor volt a fajlban.")
print(sorok)
```

Figyelem! A fájlból olvasott sorokra mindig hívd meg a `strip()` metódust, mielőtt bármit csinálsz velük.

Próbáld ki - adatsorokból saját típusú elemek

A `diakok.txt` minden sora egy diák adatait tartalmazza `nev;kor;atlag` formában. A sorból `Diak` típusú változót készítő `feldolgoz` függvényt a dataclass jegyzetből ismered:

```
from dataclasses import dataclass

@dataclass
class Diak:
    nev: str
    kor: int
    atlag: float

def feldolgoz(sor):
    reszek = sor.split(";")
    return Diak(reszek[0], int(reszek[1]), float(reszek[2]))

diakok = []
with open("diakok.txt") as be:
    for sor in be:
        diakok.append(feldolgoz(sor.strip()))

print(len(diakok), "diak adatait olvastam be.")
print("Az első:", diakok[0])
```

Az előző feladatlapon a sorokat beírtuk a programba. Most ugyanilyen sorok fájlból érkeznek – a `feldolgoz` függvényen semmit nem kellett változtatni.

Tipp: Beolvasás után mindig írd ki, hány elemet olvastál be, és mi az első elem. Ha ez a kettő stimmel, a beolvasásod nagy valószínűséggel jó.

Feladatok

2.1. feladat ★☆☆ - Névsor

Olvasd be a `nevsor.txt` sorait egy listába. Írd ki, hány név van benne, és melyik a leghosszabb név.

Ellenőrzés: 6 név; a leghosszabb `Kovacs Ferenc Pal` (17 karakter). Ha 18-at kaptál, kimaradt a `strip()`.

2.2. feladat ★☆☆ - Diákok

A fenti program segítségével számold ki a `diakok.txt` diákjainak átlagos tanulmányi átlagát.

Ellenőrzés: 10 diák, az átlaguk 3.92.

2.3. feladat ★☆☆ - A legjobb

Kinek a legjobb az átlaga?

Ellenőrzés: Toth Csilla (4.9).

2.4. feladat ★☆☆ - Nagykorúak

Írd ki a nagykorú (legalább 18 éves) diákok nevét.

Ellenőrzés: Toth Csilla, Kovacs Feri, Balogh Iren.

3. Két gyakori fájlforma

Az adatfájlok első sora gyakran nem adat. Vagy azt mondja meg, hány adatsor következik, vagy fejléc: az oszlopok neve.

Próbáld ki - darabszám az első sorban

A `termekek.txt` tartalma:

```
6
fuzet;350;12
toll;180;30
radir;120;8
vonalzo;420;5
ceruza;90;40
korzo;990;3
```

```
from dataclasses import dataclass

@dataclass
class Termek:
    nev: str
    ar: int
    darab: int

def feldolgoz(sor):
    reszek = sor.split(";")
    return Termek(reszek[0], int(reszek[1]), int(reszek[2]))

termekek = []
with open("termekek.txt") as be:
    db = int(be.readline())
    for i in range(db):
        sor = be.readline().strip()
        termekek.append(feldolgoz(sor))

print(len(termekek), "termek adatait olvastam be.")
```

A `be.readline()` egyetlen sort olvas be, és az olvasás a következő sornál folytatódik. Az első sorból szám lesz (`db`), utána a `for` ciklus pontosan ennyi sort olvas.

Kísérlet - ha az első sort is adatnak nézed

Cseréld le a `with` blokk belsejét erre, és futtasd:

```
for sor in be:
    termekek.append(feldolgoz(sor.strip()))
```

Mi történik? Keresd meg a hibaüzenetet a Hibakeresőben, aztán állítsd vissza az eredeti programot.

Próbáld ki - fejléc az első sorban

A `meresek.txt` első néhány sora:

```
nap;homerseklet;csapadek
1;12.5;0
2;14.1;0
3;9.8;4.2
```

Az első sor nem adat, ezért beolvassuk, de nem dolgozzuk fel:

```
with open("meresek.txt") as be:
    fejléc = be.readline()
    for sor in be:
        sor = sor.strip()
        # itt dolgozd fel a sort
```

Feladatok

3.1. feladat ★☆☆ - Készlet

Mennyit ér a `termekek.txt` teljes készlete (ár szorozva darabszámmal, összesen)?

Ellenőrzés: 6 termék, a készlet értéke 19230.

3.2. feladat ★☆☆ - Legdrágább

Melyik a legdrágább termék?

Ellenőrzés: `korzo` (990).

3.3. feladat ★☆☆ - Mérések

Írj `Meres` típust (`nap`, `homerseklet`, `csapadek`) és hozzá `feldolgoz` függvényt. Olvasd be a `meresek.txt` adatait a fejléc kihagyásával, és írd ki az átlaghőmérsékletet két tizedesjegyre.

Ellenőrzés: 14 mérés, az átlaghőmérséklet 11.56.

3.4. feladat ★☆☆ - Csapadék

Hány napon esett csapadék, és mennyi esett összesen? Mi történik, ha a fejléct elfelejtet kihagyni?

Ellenőrzés: 6 napon, összesen 26.0.

4. Írás fájlba

Fájlba a `print` függvénnyel is írhat. Csak előbb meg kell nyitnod a fájlt írásra, és a `print`-nek meg kell mondanod, hová írnod.

Próbáld ki

```
with open("eredmeny.txt", "w") as ki:
    print("Ez az első sor.", file=ki)
    print("Harom meg negy az", 3 + 4, file=ki)

print("Kész, nezd meg a fájlt!")
```

- `open("eredmeny.txt", "w")` – megnyitja a fájlt **írásra** (a `"w"` a `write` szó rövidítése). Ha a fájl még nem létezik, létrehozza.
- `file=ki` – ettől a `print` nem a képernyőre, hanem a fájlba ír.
- A `with` blokk végén a fájl bezárul. Ami a blokk után áll, az már újra a képernyőre megy.

Keresd meg az `eredmeny.txt` fájlt az adatfájlok mellett, és nyisd meg.

Kísérlet - mi lesz a régi tartalommal?

Írd át a programban az első sort másra, és futtasd újra. Mi van most a fájlban: a régi és az új szöveg is, vagy csak az új?

Kísérlet - ha kimarad a "w"

Töröld ki a megnyitásból a , "w" részt, és futtasd. Olvasd el a hibaüzenet utolsó sorát, és keresd meg a Hibakeresőben.

Figyelem! A "w" móddal megnyitott fájl korábbi tartalma azonnal törlődik. Soha ne nyisd meg írásra azt a fájlt, amelyből az adataidat olvasod!

Próbáld ki - olvasás és írás egy programban

Ez a program beolvassa a `szamok.txt` számait, és mindegyik kétszeresét kiírja a `duplak.txt` fájlba:

```
with open("szamok.txt") as be:
    tartalom = be.read()

with open("duplak.txt", "w") as ki:
    for resz in tartalom.split():
        print(int(resz) * 2, file=ki)
```

Ugyanazt az eredményt kiírhatod a képernyőre és fájlba is: az egyik `print` után nincs `file=ki`, a másik után van.

Tipp: Ha egy adatfájlban ékezetes betűk vannak, a megnyitáskor add meg a kódolást is: `open("adat.txt", encoding="utf-8")`. A mostani adatfájlokban nincs ékezet, ezért erre itt nincs szükség.

Feladatok

4.1. feladat ★☆☆ - Névjegy

Írj programot, amely a `nevjegy.txt` fájlba kiírja a nevedet és az osztályodat, két külön sorba.

Ellenőrzés: nyisd meg a fájlt; két sor legyen benne.

4.2. feladat ★☆☆ - Négyzetszámok

Írd ki a `negyzetek.txt` fájlba a számokat 1-től 20-ig, mindegyik mellé a négyzetét, szóközzel elválasztva.

Ellenőrzés: 20 sor; az első `1 1`, az utolsó `20 400`.

4.3. feladat ★☆☆ - Csak a párosak

Olvasd be a `szamok.txt` számait, és a párosakat írd ki a `paros.txt` fájlba, soronként egyet.

Ellenőrzés: a fájl hat sora: `12, 4, 18, 16, 22, 40`.

4.4. feladat ★☆☆ - Kitűnők

Olvasd be a `diakok.txt` adatait, és írd ki a `kitunok.txt` fájlba azoknak a nevét és átlagát, akiknek az átlaga legalább 4.5.

Ellenőrzés: három sor: `Kiss Anna 4.5, Toth Csilla 4.9, Horvath Greta 4.7`.

4.5. feladat ★☆☆ - Oda-vissza

Olvasd vissza a 4.2. feladatban készített `negyzetek.txt` fájlt egy másik programmal, és add össze a négyzetszámokat (a sorok második számát).

Ellenőrzés: 2870.

5. Lépésszámláló

Amit gyakorolsz: számok beolvasása listába, alapminták, szomszédos elemek, kiírás fájlba.

A `lepesek.txt` egy okosóra októberi adatait tartalmazza: minden sorban egy nap lépésszáma áll, a hónap első napjától az utolsóig.

5.1. feladat ★☆☆ - Beolvasás

Olvasd be az adatokat egy listába. Írd ki, hány nap adata van meg, és mennyi az első és az utolsó napi érték.

Ellenőrzés: 31 nap; az első érték 5635, az utolsó 13944.

5.2. feladat ★☆☆ - Összesen és átlagosan

Hány lépést tett meg a hónapban összesen, és mennyi a napi átlag (két tizedesjegyre)?

Ellenőrzés: összesen 337918 lépés, az átlag 10900.58.

5.3. feladat ★☆☆ - A legaktívabb nap

Hányadik napon lépett a legtöbbet, és mennyit?

Ellenőrzés: a 4. napon, 15509 lépést.

5.4. feladat ★☆☆ - Tízezres napok

Hány napon érte el a 10 000 lépést?

Ellenőrzés: 21 napon.

5.5. feladat ★☆☆ - Jó napok fájlba

Írd ki a `jo_napok.txt` fájlba azokat a napokat, amikor az átlagnál többet lépett: soronként a nap sorszáma és a lépésszám, szóközzel elválasztva.

Ellenőrzés: a fájlban 18 sor van; az első 2 13498, az utolsó 31 13944.

5.6. feladat ★☆☆ - Hétvége

Október 1-je csütörtökre esett. Mennyi a hétvégi napok (szombat és vasárnap) átlaga, és mennyi a hétköznapiaké?

Ellenőrzés: 9 hétvégi nap, átlaguk 10562.0; 22 hétköznapi, átlaguk 11039.09.

5.7. feladat ★★★ - Leghosszabb sorozat

Hány napig tartott a leghosszabb olyan sorozat, amikor minden egymást követő napon megvolt a 10 000 lépés? Melyik napon kezdődött?

Ellenőrzés: 5 napig tartott, és a 4. napon kezdődött.

5.8. feladat ★★★ - A legjobb hét

Melyik hét egymást követő napon lépett összesen a legtöbbet? Add meg a kezdőnap sorszámát és a hét napi összeget.

Ellenőrzés: a 2. napon kezdődő hét nap, összesen 89326 lépés.

6. Ügyfélszolgálat

Amit gyakorolsz: több szám egy sorban, saját típus, számolás időpontokkal, számláló lista, szomszédos elemek.

A `hivasok.txt` egy telefonos ügyfélszolgálat egy napi hívásait tartalmazza, időrendben. Egy sor egy hívás: hat szám, szóközzel elválasztva. Az első három a hívás kezdete (óra, perc, másodperc), a második három a vége. Az első három sor:

```
8 1 35 8 13 56
8 13 40 8 21 40
8 21 38 8 22 38
```

6.1. feladat ★☆☆ - Másodpercben

Írj `mpben(ora, perc, mp)` függvényt, amely megadja, hány másodperc telt el éjfél óta.

Ellenőrzés: `mpben(8, 1, 35) → 28895`, `mpben(0, 0, 59) → 59`.

6.2. feladat ★☆☆ - Beolvasás

Írj `Hivas` típust két mezővel: `kezdet` és `veg` (mindkettő másodpercben), és egy `hossz` metódussal. Olvasd be a hívásokat egy listába.

Ellenőrzés: 50 hívás; az első hossza 741 másodperc.

6.3. feladat ★☆☆ - A leghosszabb hívás

Hányadik hívás volt a leghosszabb, és meddig tartott? Írd ki percben és másodpercben is.

Ellenőrzés: a 18. hívás, 759 másodperc (12 perc 39 másodperc).

6.4. feladat ★☆☆ - Hívásidők

Mennyi az átlagos hívásidő? Hány hívás tartott 5 percnél tovább? Mennyi az összes beszélgetési idő óra, perc, másodperc formában?

Ellenőrzés: az átlag 268.7 másodperc; 16 hívás volt 5 percnél hosszabb; az összes idő 3 óra 43 perc 55 másodperc.

6.5. feladat ★☆☆ - Óránkénti forgalom

Melyik órában hány hívás kezdődött? Írd ki az eredményt a `forgalom.txt` fájlba (8 óra: 8 hívás formájú sorokkal), és a képernyőre azt, melyik órában kezdődött a legtöbb hívás.

Ellenőrzés: 8 óra: 8, 9 óra: 6, 10 óra: 7, 11 óra: 7, 12 óra: 6, 13 óra: 4, 14 óra: 5, 15 óra: 7. A legtöbb hívás 8 és 9 óra között kezdődött.

6.6. feladat ★☆☆ - Várakozók

Az ügyfélszolgálaton egyetlen ügyintéző dolgozik. Hány hívó volt kénytelen várni, vagyis hány hívás kezdődött úgy, hogy az előző hívás még nem ért véget?

Ellenőrzés: 14.

6.7. feladat ★☆☆ - A leghosszabb szünet

Mennyi volt a leghosszabb szünet, vagyis a leghosszabb idő egy hívás vége és a következő hívás kezdete között? Melyik két hívás között volt?

Ellenőrzés: 1267 másodperc (21 perc 7 másodperc), a 36. és a 37. hívás között.

6.8. feladat ★☆☆ - Hosszú hívások fájlba

Írd ki a `hosszu.txt` fájlba az 5 percnél hosszabb hívásokat: soronként a hívás sorszáma, a kezdetének órája és perce, valamint a hossza másodpercben.

Ellenőrzés: 16 sor; az első `1 8 1 741`, az utolsó `49 15 34 564`.

7. Futóverseny

Amit gyakorolsz: *fejléces fájl, saját típus, keresés, megszámlálás kategóriánként, rendezés, formázott kiírás fájlba.*

A `verseny.txt` egy futóverseny eredményeit tartalmazza. Az első sor fejléc, utána minden sor egy versenyző: rajtszám, név, kategória (`F` férfi, `N` női, `J` junior), és az idő percben és másodpercben. Az első négy sor:

```
rajtszam;nev;kategoria;perc;masodperc
42;Papp Tamas;F;42;42
20;Meszaros Bence;F;48;3
51;Papp Reka;N;52;28
```

7.1. feladat ★★☆☆ - Beolvasás

Írj `Futo` típust az öt mezővel és egy `ido` metódussal, amely az időt másodpercben adja vissza. Olvasd be a versenyzőket a fejléc kihagyásával.

Ellenőrzés: 36 versenyző; az első Papp Tamas, az ideje 2562 másodperc.

7.2. feladat ★★☆☆ - Időpont szépen

Írj `ido_szoveg(mp)` függvényt, amely a másodpercben megadott időt `perc:masodperc` formájú szöveggé alakítja. A másodperc mindig két számjegy legyen.

Ellenőrzés: `ido_szoveg(2412) → 40:12`, `ido_szoveg(3243) → 54:03`.

7.3. feladat ★★☆☆ - A győztes

Ki nyerte a versenyt, és mennyi idő alatt?

Ellenőrzés: Kiss Petra, 40:12.

7.4. feladat ★★☆☆ - Létszámok

Hány versenyző indult az egyes kategóriákban?

Ellenőrzés: F: 15, N: 14, J: 7.

7.5. feladat ★★☆☆ - Kategóriagyőztesek

Ki nyert az egyes kategóriákban? Írj hozzá függvényt, amely megkapja a versenyzők listáját és egy kategóriát, és visszaadja a kategória legjobbját.

Ellenőrzés: F: Molnar Dani (40:18); N: Kiss Petra (40:12); J: Meszaros Reka (46:38).

7.6. feladat ★★☆☆ - Átlagidő

Mennyi az átlagos idő? Hány versenyző futott az átlagnál jobbat?

Ellenőrzés: 3105.36 másodperc, vagyis 51:45; 17 versenyző volt ennél gyorsabb.

7.7. feladat ★★☆☆ - Keresés

Olvass be egy rajtszámot a billentyűzetről. Írd ki a versenyző nevét, idejét és azt, hányadik helyen végzett összesítésben. Ha nincs ilyen rajtszám, ezt írd ki: `Nincs ilyen rajtszam.`

Ellenőrzés: 71 → Farkas Oliver, 45:11, 10. hely; 52 → Racz Greta, 54:03, 21. hely; 1 → `Nincs ilyen rajtszam.`

7.8. feladat ★★☆☆ - Eredménylista

Írd ki az `eredmeny.txt` fájlba a verseny végeredményét, a leggyorsabbtól a leglassabbig, ilyen sorokkal:

1. Kiss Petra (N) 40:12

Ellenőrzés: 36 sor. Az első három: 1. Kiss Petra (N) 40:12, 2. Molnar Dani (F) 40:18, 3. Kiss Levente (F) 41:29. Az utolsó: 36. Szabo Flora (N) 64:41.

7.9. feladat ★★☆☆ - Dobogósok

Írd ki a `dobogo.txt` fájlba mindhárom kategória első három helyezettjét.

Ellenőrzés: F: Molnar Dani, Kiss Levente, Papp Tamas. N: Kiss Petra, Balogh Lilla, Juhasz Bori. J: Meszaros Reka, Varga Csilla, Kovacs Bence.

8. Iskolai büfé

Amit gyakorolsz: saját típus, különböző értékek kigyűjtése, összesítés termékenként, rendezés, fájl írása és visszaolvasása. Ez a jegyzet legnehezebb fejezete.

A `vasarlasok.txt` egy iskolai büfé egy heti eladásait tartalmazza. Egy sor egy eladott tétel: a nap, a termék neve, a darabszám és az egységár. Ugyanaz a termék egy napon többször is szerepelhet. Az első négy sor:

```
hetfo;szendvics;5;650
hetfo;alma;4;150
hetfo;kakaos csiga;2;420
hetfo;pogacs;7;280
```

8.1. feladat ★☆☆ - Beolvasás

Írj `Tétel` típust a négy mezővel és egy `ertek` metódussal (darabszám szorozva az egységárral). Olvasd be a tételeket.

Ellenőrzés: 56 tétel; az első tétel értéke 3250.

8.2. feladat ★☆☆ - Heti bevétel

Mennyi volt a büfé heti bevétele?

Ellenőrzés: 96810 Ft.

8.3. feladat ★☆☆ - A legnagyobb tétel

Melyik volt a legnagyobb értékű tétel?

Ellenőrzés: hétfőn 9 szendvics, 5850 Ft.

8.4. feladat ★☆☆ - Napi bevétel

Írj `napi_bevetel(lista, nap)` függvényt. Írd ki mind az öt nap bevételeit, és azt, melyik napon volt a legnagyobb.

Ellenőrzés: hetfo 18370, kedd 23590, szerda 13300, csutortok 21320, pentek 20230. A legjobb nap a kedd.

8.5. feladat ★★ - Termékek

Hányféle terméket árul a büfé? Gyűjtsd ki a termékek nevét úgy, hogy mindegyik csak egyszer szerepeljen.

Ellenőrzés: 8-féle: szendvics, alma, kakaos csiga, pogacs, muzli szelet, viz, tea, joghurt.

8.6. feladat ★★ - Összesítés termékenként

Írj `Osszesites` típust: `termek`, `darab`, `bevetel`. Készíts minden termékhez egy összesítést: hány darabot adtak el belőle a héten, és mennyi bevételt hozott. Melyik termékből fogyott a legtöbb darab? Melyik hozta a legtöbb bevételt?

Ellenőrzés: a legtöbb darab a pogácsából fogyott (63); a legtöbb bevételt a szendvics hozta (24050 Ft).

8.7. feladat ★★ - Összesítés fájlba, és vissza

Írd ki az összesítést az `osszesites.txt` fájlba, bevétel szerint csökkenő sorrendben, `termek;darab;bevetel` formában. Utána ellenőrizd a saját munkádat: egy másik programmal olvasd vissza a fájlt, és add össze a bevételeket.

Ellenőrzés: az első sor `szendvics;37;24050`, az utolsó `tea;25;7500`. A visszaolvasott bevételek összege 96810, vagyis megegyezik a 8.2. feladat eredményével.

8.8. feladat ★★★ - Mi nem fogyott pénteken?

Melyik termékekből nem adtak el egyet sem pénteken?

Ellenőrzés: alma és tea.

Hibakereső: mi lehet a baj?

A Python hibaüzenetének mindig az **utolsó sorát** olvasd el először. Keresd meg a táblázatban azt, amit tapasztalsz.

Amit látsz	Mit jelent?	Mit tegyél?
<code>FileNotFoundError: [Errno 2] No such file or directory: 'szamok.txt'</code>	A Python nem találja a fájlt.	Jól írtad a fájl nevét? Jó mappában van?
<code>ValueError: invalid literal for int() with base 10: 'nap'</code>	Az <code>int()</code> olyan szöveget kapott, amely nem szám.	Kihagytad a fejléct? Van üres sor a fájl végén? Jó darabot alakítasz át?
<code>IndexError: list index out of range</code> a <code>feldolgoz</code> függvényben	A sorban kevesebb darab van, mint amire számítasz.	Az első sor darabszám vagy fejléc? Jó elválasztójelet adtál a <code>split</code> -nek?
Üres sorok jelennek meg a kiírásban, vagy eggyel hosszabb minden szöveg	A sorok végén ott maradt a sortörés.	<code>sor.strip()</code>
<code>io.UnsupportedOperation: not writable</code>	Olvasásra megnyitott fájlba próbálsz írni.	A megnyitásnál kell a <code>"w"</code> .
<code>ValueError: I/O operation on closed file</code>	A <code>with</code> blokkon kívül próbálsz használni a fájlt.	Ellenőrizd a behúzást: az olvasás vagy írás a blokkon belül legyen.
A kimeneti fájlban csak az utolsó sor van	A fájlt a cikluson belül nyitod meg, így minden körben törlődik.	A <code>with open(..., "w")</code> kerüljön a ciklus elé.
A képernyőn megjelenik az eredmény, a fájlban nem	A <code>print</code> hívásából kimaradt a <code>file=ki</code> .	Írd oda minden olyan <code>print</code> -hez, amelyik a fájlba szól.
A kimeneti fájlt nem találod	A fájl ott jön létre, ahol a program fut.	Ugyanott keresd, ahol az <code>itt_vagyok.txt</code> volt.
Az adatfájlod kiürült	Megnyitottad <code>"w"</code> móddal.	Másold be újra az eredeti fájlt, és javítsd a fájlnevet a programban.

Összefoglaló táblázat (puska)

Minta	Jelentés
<code>with open("adat.txt") as be:</code>	fájl megnyitása olvasásra; a blokk végén bezárul
<code>tartalom = be.read()</code>	a teljes tartalom egy sztringben
<code>tartalom.split()</code>	darabolás szóközöknél és sortöréseknél
<code>for sor in be:</code>	olvasás a fájl végéig, soronként
<code>sor = sor.strip()</code>	a sor végi sortörés levágása
<code>be.readline()</code>	egyetlen sor beolvasása (darabszám vagy fejléc)
<code>db = int(be.readline())</code>	darabszám az első sorban
<code>with open("ki.txt", "w") as ki:</code>	fájl megnyitása írásra (a régi tartalom törlődik)
<code>print(x, y, file=ki)</code>	írás a fájlba

Figyelem! Három dolog, amit soha ne felejts el: a fájlból olvasott sor végén sortörés van, vágd le a `strip()` metódussal; a fájlból minden szöveggként érkezik, a számokat át kell alakítanod; az adatfájlt soha ne nyisd meg "w" móddal.

Segítség a feladatokhoz

Csak akkor olvasd el egy feladat segítségét, ha már legalább öt percig próbálkoztál vele. A segítség nem megoldás: azt mondja meg, merre indulj.

5. fejezet

- **5.1.** Az 1. fejezet programja: `be.read()`, `split()`, és minden darabra `int()`.
- **5.3.** Itt az index kell, ezért `range(len(lepesek))` szerint járd be a listát.
- **5.5.** Előbb az átlag kell. Utána nyiss meg egy fájlt írásra, és a ciklusban a feltételnek megfelelő napokat írd bele: `print(i + 1, lepesek[i], file=ki)`
- **5.6.** Ha az 1. nap csütörtök, akkor a 3. nap szombat, a 4. vasárnap, és ez hétnaponként ismétlődik. A nap sorszámának 7-tel vett maradéka tehát szombaton 3, vasárnap 4.
- **5.7.** Vezess egy `hossz` számlálót: ha az aznapi érték legalább 10 000, növeld; ha nem, nullázd. Közben jegyezd meg az eddigi legnagyobb hosszt, és azt, hol kezdődött az a sorozat.
- **5.8.** Minden lehetséges kezdőnapra add össze a hét egymást követő nap értékét (ciklus a ciklusban), és keresd a legnagyobb összeget. Milyen `range()` kell a kezdőnap indexéhez?

6. fejezet

- **6.2.** Soronként olvass. Egy sor `split()` után hat darabból áll; mindegyiket `int()` alakítja számmá. A hat számból az `mpben` függvénnyel készíts egy `Hivas` elemet.
- **6.3.** Maximumkiválasztás a `hossz()` értékek szerint. A perc: `hossz // 60`, a másodperc: `hossz % 60`.
- **6.4.** Az óra: `ossz // 3600`, a perc: `ossz % 3600 // 60`, a másodperc: `ossz % 60`.
- **6.5.** `darab = [0] * 24` – minden hívásnál `darab[h.kezdet // 3600] += 1`. A fájlba csak azokat az órákat írd ki, ahol a darabszám nem nulla.
- **6.6.** Megszámolás: az `i`-edik hívás `kezdet` mezőjét hasonlítsd az `i - 1`-edik `veg` mezőjéhez. A ciklus 1-től induljon.
- **6.7.** Maximumkiválasztás a `hivasok[i].kezdet - hivasok[i - 1].veg` különbségekre.
- **6.8.** A kezdet órája `kezdet // 3600`, a perce `kezdet % 3600 // 60`.

7. fejezet

- **7.1.** A fejléchez egy `be.readline()` a ciklus előtt. A `feldolgoz` függvényben `sor.split(";")`; a három számot `int()` alakítja át.
- **7.2.** A perc `mp // 60`, a másodperc `mp % 60`. Ha a másodperc 10-nél kisebb, tegyél elé egy "0" karaktert. (Rövidebben: f-stringben a `{x:02d}` két számjegyre egészíti ki a számot.)
- **7.3.** Minimumkiválasztás az `ido()` szerint.
- **7.4.** Három számláló, vagy egy függvény: `letszam(lista, kat)`.
- **7.5.** Minimumkiválasztás, de csak az adott kategória versenyzői között. Vigyázz: a kezdőérték is ebből a kategóriából való legyen (például indulj `None` értékről, és az első megfelelő versenyzőt fogadd el).
- **7.6.** Az `ido_szoveg` függvénynek az átlag egész részét add át: `int(atlag)`.
- **7.7.** Keresés a rajtszám alapján. A helyezéshez számold meg, hány versenyző ideje jobb az övénel; a helyezés ennél eggyel nagyobb.
- **7.8.** Rendezés kulcsfüggvénnyel (amely az `ido()` értékét adja vissza), utána egy ciklus ír a fájlba.
- **7.9.** A már rendezett listán menj végig kategóriánként, és az első három megfelelő versenyzőt írd ki. Számláló kell hozzá.

8. fejezet

- **8.1.** A `feldolgoz` függvényben `sor.split(";")`, a két számot `int()` alakítja át.
- **8.4.** A függvényben feltételes összegzés: csak azokat a tételeket add össze, amelyeknek a `nap` mezője egyezik. A napok nevét tedd egy listába, és ciklussal hívd meg rájuk a függvényt.
- **8.5.** Egy terméknevet csak akkor tegyél az új listába, ha még nincs benne (előző feladatlap, 5.6. és 9.4. feladat).
- **8.6.** Készíts minden terméknevhez egy `0sszesites` elemet nullákkal. Írj `keres(lista, nev)` függvényt, amely megadja a termék indexét. Menj végig a tételeken: keresd meg a termék indexét, és növeld annak az elemnek a `darab` és a `bevetel` mezőjét.
- **8.7.** Rendezés a `bevetel` mező szerint, `reverse=True` beállítással. A kiíráshoz f-sztring: `f"{o.termek};{o.darab};{o.bevetel}"`. A visszaolvasáshoz új `feldolgoz` függvény kell.
- **8.8.** Minden termékre dönts el (eldöntés), volt-e olyan tétel, amelynek a napja `pentek`, a terméke pedig az adott termék. Azokat írd ki, amelyekre nem volt.